

Comprehensive Guide: Bus Reservation System

1. What the Project is Doing

The **Bus Reservation System** is a complete, end-to-end desktop application for booking bus tickets. It simulates a real-world booking kiosk or agent portal where a user can:

1. **Register and Log in:** Securely authenticate using their credentials.
 2. **Select Travel Details:** Pick a travel date, destination route, travel time, bus type (e.g., AC, Sleeper), and input the number of passengers.
 3. **Interactive Seat Selection:** View a visual grid of bus seats. The system intelligently disables seats that are already booked for that specific route and time. The user can click to reserve the exact seats they want.
 4. **Meal Pre-booking (Add-ons):** Optionally select food items (e.g., Sambar Rice, Dosa) to be served during the journey.
 5. **Invoice Generation:** Generate a final, non-editable booking summary containing all travel details, passenger names, seat numbers, and total costs (tickets + meals).
-

2. Technology Stack & Architecture

The Tech Stack

- **Language:** Java (JDK 17+)
- **User Interface:** Java Swing & AWT (Java's native desktop GUI toolkit)
- **Database:** MySQL
- **Database Connector:** JDBC (MySQL Connector/J 8.0.33)

The "Backend" Explained (2-Tier Architecture)

A common question is: *Where is the backend, and why isn't a framework like Spring Boot used?*

This project does **not** use a modern 3-Tier Architecture (Frontend Web App ↔ Spring Boot REST API ↔ Database). Instead, it uses a classic **2-Tier Monolithic Desktop Architecture**.

In this architecture, the **Frontend (UI)** and the **Backend (Business Logic)** are tightly bundled together in the exact same Java program:

1. The UI is drawn directly on the operating system's screen using Java Swing.
2. When a button is clicked, it triggers a local Java function (an `ActionListener`).
3. That local Java function directly opens a socket connection to the **MySQL Database** via JDBC on port 3306.
4. It executes raw SQL queries directly from the desktop application, receives the response, and updates the local UI.

Pros of this approach: Very fast to build for simple, local kiosk applications. **Cons of this approach:** Highly insecure for public deployment, as the database credentials must be hardcoded inside the desktop application's source code. A modern rebuild would decouple this by placing Spring Boot in the middle to handle database logic securely.

3. Database Schema

The system relies on a local MySQL database named `busbookingsystem` with four core tables:

1. **users** : Handles authentication (columns: `id` , `username` , `password` , `created_at`).
 2. **bookings** : The master record for the invoice (columns: `id` , `route` , `travel_date` , `time` , `contact` , `passenger_names` , `total_price`).
 3. **seat_bookings** : Prevents double-booking by tracking individual reserved seats (columns: `id` , `route` , `travel_date` , `time` , `seat_number` , `passenger_names` , `contact`).
 4. **meals** : A catalog of available meals (columns: `id` , `meal_name` , `image_path`).
-

4. The Overall Application Flow (Code Level)

The application utilizes Java's `CardLayout` manager, which allows different `JPanel` screens to be swapped in and out of a single main `JFrame` window seamlessly.

1. **MainApp.java (Entry Point)**: Initializes the main `JFrame` and the `CardLayout` . Displays a splash screen with a "Book Ticket" button.
 2. **RegistrationLoginApp.java** : Triggered by the home screen. Provides split tabs for Login and Registration. It executes `INSERT INTO users` for registration and `SELECT * FROM users` for login.
 3. **TravelBooking.java** : Upon successful login, this screen captures standard travel inputs (dropdowns for route/time, text fields for dates). When proceeding, it dynamically asks for passenger names based on the inputted passenger count using a `JOptionPane` loop.
 4. **MultiSeatBookingSwing.java** : The core seat engine. Before rendering the seats, it queries the `seat_bookings` table to check which seats are already occupied for the selected date and time. Occupied seats are rendered as disabled, red `JToggleButton` components. Available seats can be clicked and toggled green.
 5. **MealSelectionSwing.java** : Displays dynamic `JCheckBox` items based on the `meals` database table, adding the cost to the running total.
 6. **BookingSummarySwing.java** : The final transition. It receives the massive state payload (all selected variables) through its constructor and renders a read-only receipt using a vertical `BoxLayout` .
-

5. Development Hurdles & Applied Fixes

When the repository was first cloned, it was fundamentally broken. Here is how it was repaired to make it functional:

- **Missing Source Code Reconstructed**: The author only uploaded compiled `.class` files, but forgot critical inner-class files (e.g., `MainApp$1.class` used for event listeners). We resolved this by downloading a decompiler (`cfr.jar`), reverse-engineering the bytecode back into `.java` source code, manually patching the missing anonymous classes with standard `JPanel` objects, and recompiling the project from scratch.
- **Database Network Configuration**: The application initially failed with a "Communications link failure". This was because the local MySQL service was running with `skip-networking` enabled. We forcefully restarted MySQL via Homebrew with networking enabled on port 3306.
- **Hardcoded Passwords Bypassed**: The `DBConnection.class` was hardcoded to expect the password `teddybear@2006` . We executed a root password reset script to clear the local MySQL password and rewrote `DBConnection.java` to connect successfully.
- **Schema Mismatches Fixed**: The author's provided database setup file created a table named `bookseats` , but their compiled Java code attempted to insert data into a table named `seat_bookings` . We manually created the correct `seat_bookings` table schema so the application could successfully save reservations.